Team 11
Voting System

Software Design Document

Name (s): Ben Pullis (pulli068), Adam Douiri (douir002), Quang Pham (pham0327), Joe Anderson (and10287)
Lab Section: 012

Date: 10/17/2025

Table of Contents

1. Introduction **3**
1.1 Purpose 3
1.2 Scope 3
1.3 Overview 3
1.4 Reference Material 3
1.5 Definitions and Acronyms 3

2. System Overview **4**

3. System Architecture **5**
3.1 Architectural Design 5
3.2 Decomposition Description 7
3.3 Design Rationale 7

4. Data Design **8**
4.1 Data Description 8
4.2 Data Dictionary 8

5. Component Design **10**

6. Human Interface Design **12**
6.1 Overview of User Interface 12
6.2 Screen Images 13
6.3 Screen Objects and Actions 14

7. Requirements Matrix **15**

8. Appendices **17**

1. Introduction

1.1 Purpose

This software design document describes the architecture and system design of a Voting System that can perform STV and Plurality voting. The intended audience of this SDD is all stakeholders involved with the Voting System.

1.2 Scope

The software will allow election officials with existing ballot files to run an election, either in the form of STV or Plurality, and get results as well as an audit report for that election. Users of the software will be able to learn it within fifteen minutes, will have a comfortable user interface within the terminal, and will have access to extra resources such as a help window.

1.3 Overview

This document will begin by providing an overview of our system, covering the functionality, design, and context of our system. It will then cover the architecture of our system, particularly the way modules within our system interact with one another, and the responsibilities of our subsystems. Then we will discuss the way in which we organize the data in our system, including the unique data structures we come up with. In the component design section, we will delve deeper into how each component within our system functions. In the Human Interface Design, we will discuss the design of our user interface, including a rough sketch of our proposed UI. Finally, in the Requirements Matrix section, we will cross-reference our components back to what was discussed in our SRS document.

1.4 Reference Material

SRS

Mermaid Documentation

1.5 Definitions and Acronyms

N/A

2. SYSTEM OVERVIEW

Context:

- The voting system assumes that the ballot files are already somewhere in the file system, and that the user is aware of where they are located and the names of the files.
- The voting system assumes that the user does NOT know how to use command line arguments, but will know how to run a program that provides a user interface.

Functionality:

User Interface: Text-based user interface that should be learnable by a user within 15 minutes.

User Input: User can input the number of seats for the election, the algorithm to be used for the election (either STV or Plurality), and the location of the ballots.

Ballot Reader: File processing of the ballots that are provided by the user input. Converts them into a Ballot object that can be used for the voting algorithms.

STV/Plurality Voting Algorithms: Two primary voting algorithms, chosen by User Input. The STV Voting Algorithm uses a “droop quota” and multiple rounds of tabulation to handle elections. The Plurality Voting Algorithm determines election winner(s) by popularity.

Ballot Shuffling: In addition to the User Input, the user has the choice as to whether or not ballots should be shuffled before the election takes place.

Election Details Display: Extension of the user interface, there will be a dedicated UI to display election details.

Help Window: Extension of the user interface, there will be a dedicated help window for the user. This help window can be opened at any time when the election is not running.

Audit: At the end of every STV election, an audit report will be created in the form of a .txt file.

Design:

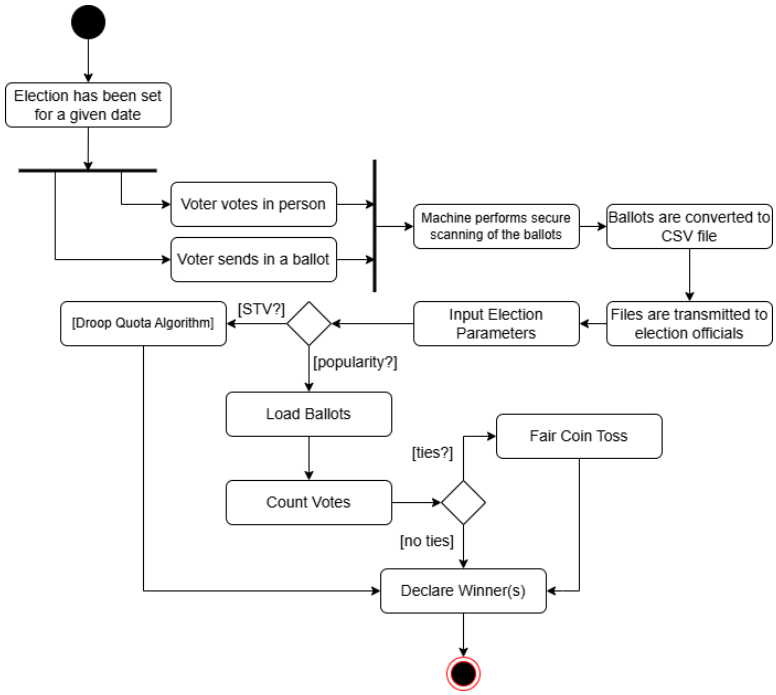
Class Design: The design is split into 7 class components. Our class design is primarily focused on the *IElection* interface, which generates the FileHandler, HelpWindow, and AuditReport classes, and stores the Ballots after they’re converted by the FileHandler.

3. System Architecture

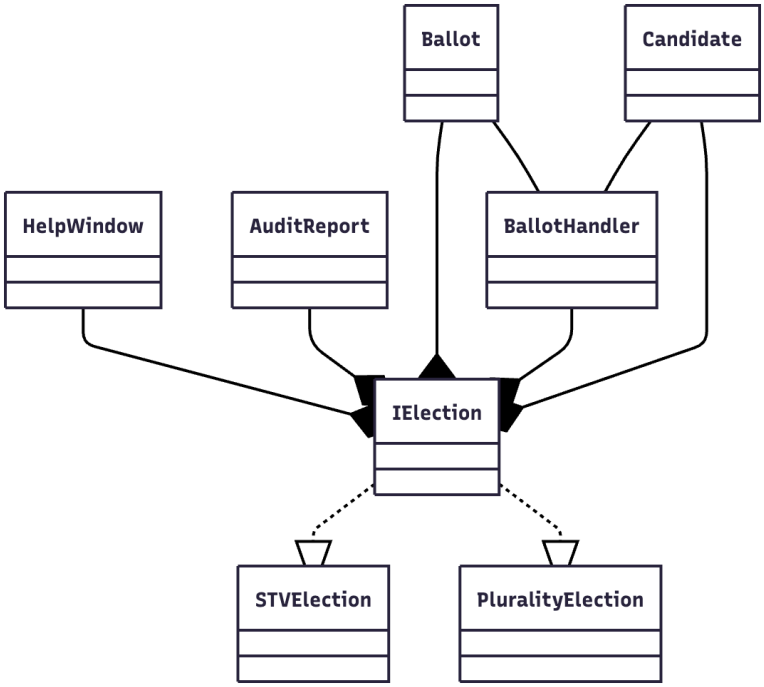
3.1 Architectural Design

Activity Diagram (Popularity): When an election is set, voters will cast their ballots. A machine securely scans them, and the ballots are converted to CSV format. The files are transmitted to election officials, who then input the election parameters (in this case, it is a popularity election). The ballots are loaded from the CSV file(s), counted, and then checked for ties. If there are ties, they are resolved using a fair coin toss. The winners are declare,d and the system ends.

The diagram is shown in the figure to the right.



Subsystems of the Election System:



Responsibility of Each Subsystem:

- **IElection**: This will be an abstract class with virtual methods allowing concrete Election types to be realized from IElection. Since this is the center of the system, it will have the responsibility of running the election and calling functions from other classes when the election needs data moved around. This class will also hold any similar attributes to all Election types. This way, we follow good OOP design by sharing functionality whenever possible

- **STVElection**: This is a concrete implementation of IElection that is created when the user wants to run an STV Election. It will contain all the logic to properly run an STV Election while also being able to access anything from its parent IElection.

- **PluralityElection**: This is a concrete implementation of IElection that is created when the user wants to run a Plurality Election. It will contain all the logic to properly run a Plurality Election while also being able to access anything from its parent IElection.

- **BallotHandler**: The purpose of the BallotHandler is to handle Ballot functionality. This includes the process of handling files to create Ballots and Candidates as well as shuffling ballots whenever needed.

- **Ballot**: The Ballot's job is to house information about a particular person's vote. This includes the candidates and what that person ranked them. The election will need to access the private variables of the ballots, so getters and setters will allow that to happen.

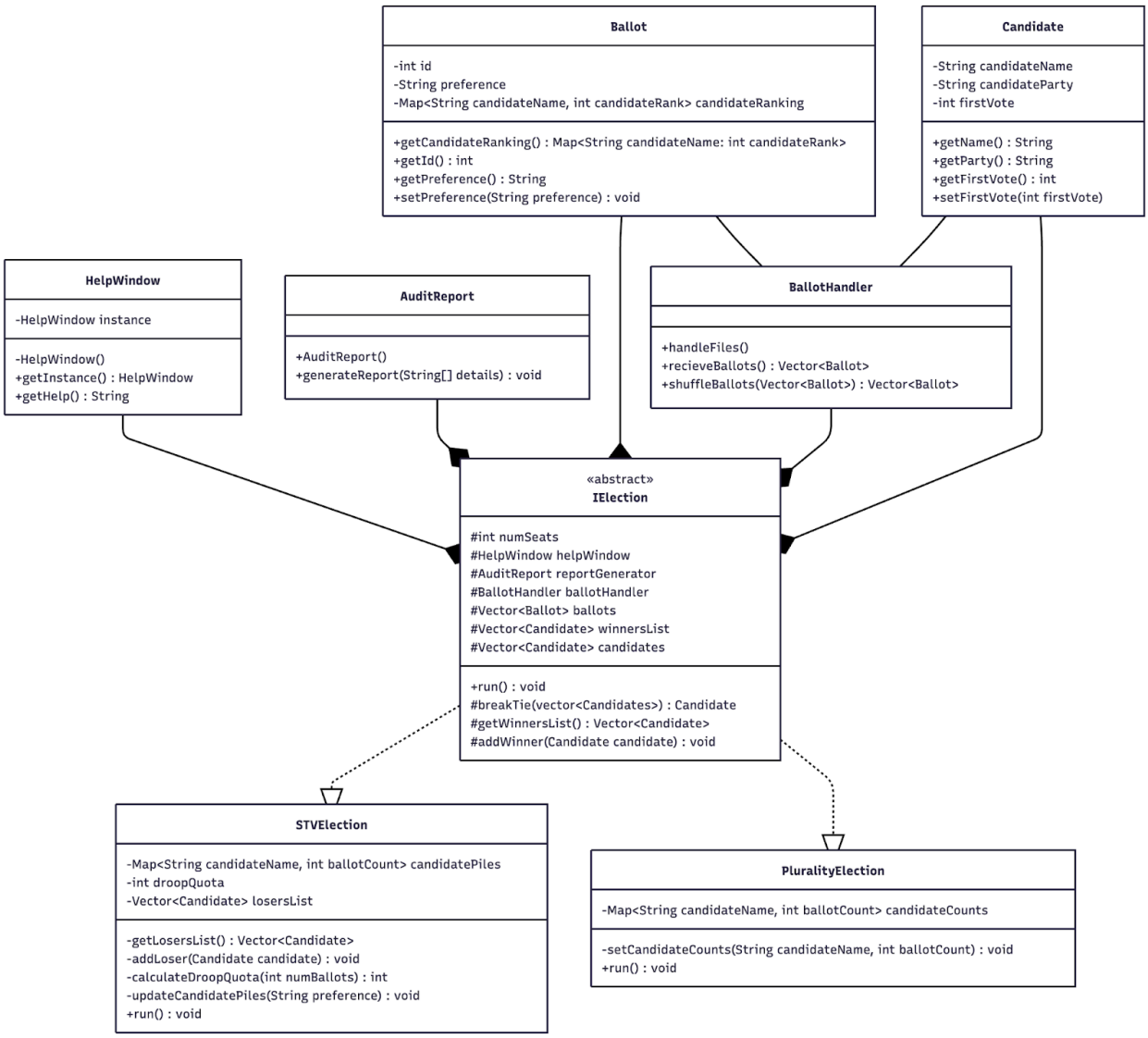
- **Candidate**: The Candidate houses information on the Candidate, including name, party, and time of first vote received for tie-breaking purposes. Getters and setters will also be used to allow access to the election.

- **HelpWindow**: The election will have one instance of a help window. Whenever a user requests help, the help window will print a set of instructions to the screen, guiding the user.

- **AuditReport**: The audit report will be used at the end of the election. The election will pass its information, and the AuditReport will generate a .txt file containing all of the election's details.

3.2 Decomposition Description

Methods, Variables, and Relationships Between Subsystems in the Voting System:



3.3 Design Rationale

The architecture we chose allows us to follow common Object Oriented Design. This is done by breaking down the Election System into a variety of subsystems represented as classes that each have unique functionality.

One of the choices we made was to start with an abstract interface called IElection. This class will never get instantiated in the code, but the subclasses that derive from it, STVElection and PluralityElection, will get instantiated. This is good because the interface can hold common functionality while letting the derived election classes implement only the functionality unique to them. This also makes it possible to add more election types in the future, which means our design confines to the "O" of the SOLID Principles.

We also have a lot of composition involved. Classes including Ballot, Candidate, BallotHandler, AuditReport, and HelpWindow are all composed by IElection because an election "has" all of those things. These classes all encapsulate functionality specific to them that can be accessed by the election whenever needed.

Finally, there is an association relationship between BallotHandler and the classes Ballot and Candidate. This is because the BallotHandler is the class responsible for creating all of the instances of these classes.

4. Data Design

4.1 Data Description

The information domain of the Voting System is transformed into structured data objects that can be processed by the voting algorithms. The primary entities include Ballots, Candidates, and election parameters, which are managed through object-oriented data structures. Ballots are stored in a Vector<Ballot>, with each Ballot object containing a Map<String, int> for candidate rankings. Candidates are represented as Candidate objects with attributes name, party, and first vote for tie-breaking. The election algorithm type (STV or Plurality) is determined by the instantiation of either STVElection or PluralityElection, which inherit from the abstract IElection class, based on the election parameters. All data is managed in-memory within the system.

4.2 Data Dictionary

- **AuditReport**
  - Methods:
    - void generateReport(String[] details)
- **Ballot**
  - Attributes:
    - int id
    - String preference
    - Map<String, int> candidateRanking
  - Methods:
    - Map<String, int> getCandidateRanking()
    - int getId()

        - String getPreference()
        - void setPreference(String preference)
- **BallotHandler**
    - Methods:
        - Vector<Ballot> handleFiles()
        - Vector<Ballot> receiveBallots()
        - Vector<Ballot> shuffleBallots(Vector<Ballot>)
- **Candidate**
    - Attributes:
        - String candidateName
        - String candidateParty
        - int firstVote
    - Methods:
        - String getName()
        - String getParty()
        - int getFirstVote()
        - void setFirstVote(int firstVote)
- **HelpWindow**
    - Attributes:
        - HelpWindow instance
    - Methods:
        - HelpWindow getInstance()
        - String getHelp()
- **IElection**
    - Attributes:
        - int numSeats
        - HelpWindow helpWindow
        - AuditReport reportGenerator
        - BallotHandler ballotHandler
        - Vector<Ballot> ballots
        - Vector<Candidate> winnersList
        - Vector<Candidate> candidates
    - Methods:
        - void run()
        - Candidate breakTie(Vector<Candidate>)
        - Vector<Candidate> getWinnersList()
        - void addWinner(Candidate candidate)
- **PluralityElection**
    - Attributes:
        - Map<String, int> candidateCounts
    - Methods:
        - void setCandidateCounts(String candidateName, int ballotCount)
        - void run()
- **STVElection**
    - Attributes:
        - Map<String, int> candidatePiles

        - int droopQuota
        - Vector<Candidate> losersList
    - Methods:
        - Vector<Candidate> getLosersList()
        - void addLoser(Candidate candidate)
        - int calculateDroopQuota(int numBallots)
        - void updateCandidatePiles(String preference)
        - void run()

5. Component Design

Candidate:

getName()

> Getter for the name of the candidate. Takes in no parameters. Returns a String containing the candidate's name.

getParty()

> Getter for the party of the candidate. Takes in no parameters. Returns a String containing the candidate's party.

getFirstVote()

> Getter for the first vote of the candidate. Takes in no parameters. Returns an int containing the first vote for the candidate.

setFirstVote(int firstVote)

> Setter for the first vote of the candidate. Takes in a parameter “firstVote” of type int. Does not return (void function).

Ballot:

getCandidateRanking()

> Getter for the ranking of candidates on an individual ballot. Takes in no parameters. Returns a map of candidate rankings of type Map<String, int>.

getId()

> Getter for the ID of the ballot. Takes in no parameters. Returns the ID of the ballot of type int.

getPreference()

> Getter for the preference of the ballot (the index of the ballot's choice candidate for an STV election). Takes in no parameters. Returns the preference as type int.

setPreference()

> Setter for the preference of the ballot. Takes in the preference integer. Does not return (void function).

BallotHandler:

handleFiles()

> Helper function for receiveBallots() that does the file processing of the ballots. Takes in no parameters. Does not return (void function).

recieveBallots()

> Allows the user to input their ballot location and performs file processing. Takes in no parameters. Returns the processed ballots of type Vector<Ballot>.

shuffleBallots()

> Performs the ballot shuffle using the random algorithm from the C standard. Takes in a parameter “ballots” of type Vector<Ballot>. Returns the shuffled version of the ballot of type Vector<Ballot>.

HelpWindow:

HelpWindow()

> Private constructor method that ensures that only one instance of this class exists (Singleton).

getInstance()

> Getter for the Singleton instance. Returns the single HelpWindow object.

getHelp()

> Prompts the user with a menu of options on what they need help with. Returns a string that will be output to the User Interface when they make their selection.

AuditReport:

AuditReport()

> Constructor method for the AuditReport, called by the IElection interface.

generateReport()

> Generates a report audit.txt file based on information provided. Takes in the parameter String[]details containing the details. Does not return (void function).

IElection:

run()

> Runs the election, begins by prompting the user for user input before creating other classes and performing other methods. Essentially, the main method of our system.

breakTie()

> Performs a tie breaker based on a coin flip. Takes in a parameter containing the list of candidates with a tie. Returns the candidate that won the coin flip (of type Candidate).

getWinnersList()

> Getter for the winners list. Does not take in any parameters. Returns a list of all of the winners as type Vector<Candidate>.

addWinner()

> Adds a winner to the winners list. Takes in a parameter Winner of type Candidates. No return (void function).

STVElection:

getLosersList()

> Getter for the losers list. Does not have any parameters. Returns a list of every loser of type Vector<candidate>

addLoser()

> Adds a loser to the losers list. The parameter of type Candidate is used to add the specified loser. No return (void function).

calculateDroopQuota()

> Calculates the Droop Quota using this equation: | numberOfBallots /

(numberOfSeats) | + 1.

updateCandidatePiles()

> Updates the candidate pile for each round of the STV election. Uses the parameter "preference" to determine where ballots are redistributed after ballot piles of losing/winning candidates are eliminated. No return (void function).

run()

> Overrides IElection run() method, see run() in IElection for basic functionality. The STV election has ballots where voters can rank multiple candidates. Runs through the STV election process by first calculating the Droop Quota (see calculateDroopQuota() method for more information), and then performing multiple rounds of eliminating/winning candidates based on ballot preferences until the election is concluded (see the other STVElection() methods for more information).

PluralityElection:

setCandidateCounts()

> Setter for the number of votes that each candidate received. Takes in the Candidate and the number of ballots received as parameters. No return (void function).

run()

> Overrides IElection run() method, see run() in IElection for basic functionality. Performs the plurality voting algorithm, which is an election system where everyone can only vote for one candidate, and it takes the candidate that has the most votes and assigns them as the winner.

6. Human Interface Design

6.1 Overview of User Interface

The voting system provides a command-line interface (CLI) designed for use by election officials. The interface allows users to become familiar with how to use the system within 15 minutes of training. Through a sequence of text-based prompts and output, the user can carry out every stage of the election process.

When the program starts, the main interface is displayed. The system prompts the user to enter the following in sequence:

1. File Name Input
2. Number of Seats
3. Voting Algorithm
4. Disable Shuffle (STV only)
5. Help
6. Close Program

When the help window is displayed, the screen provides:

- Instructions on how to operate the system.

- Descriptions of each input field and valid input options.
- Details about STV and Plurality algorithms.

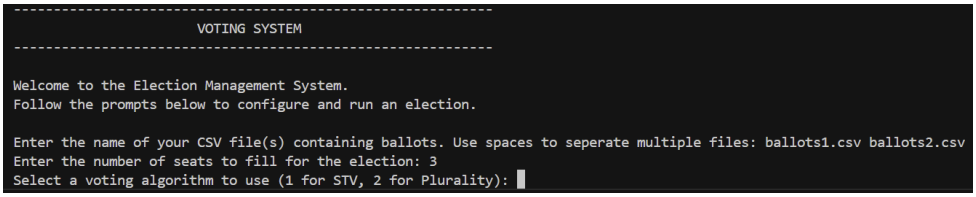
After the election completes, the system displays summary information. This includes:

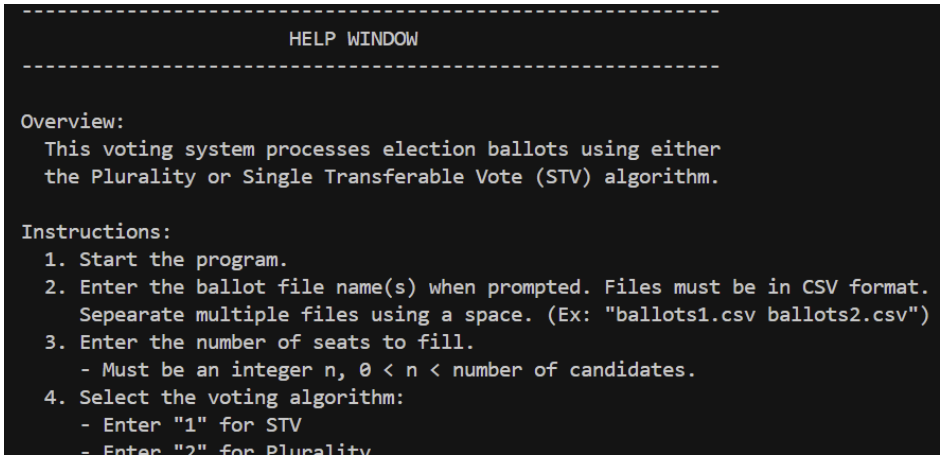
- Election type (STV or Plurality)
- Number of ballots processed
- Number of seats filled
- Number of candidates
- Election results

For plurality elections, the system displays each candidate and their percentage of votes. For STV elections, the system displays the order in which candidates were elected and eliminated.

If a processing error occurs, an error screen is displayed, which shows a descriptive error message identifying the issue.

6.2 Screen Images





```
------------------------------------------------------------
                    ELECTION RESULTS
------------------------------------------------------------

Election Type:       Plurality
Number of Ballots:   1200
Number of Seats:     3
Number of Candidates: 7

------------------------------------------------------------
Results:
------------------------------------------------------------
Candidate Name         | Party    | Votes | Percentage
------------------------------------------------------------
Omar Fateh             | DFL      | 420   | 35%
Jacob Frey             | DFL      | 310   | 25.83%
Jazz Hampton           | DFL      | 260   | 21.67%
Brenda Short           | DFL      | 140   | 11.67%
DeWayne Davis          | DFL      | 70    | 5.83%

------------------------------------------------------------
Elected Candidates:
  1. Omar Fateh
  2. Jacob Frey
  3. Jazz Hampton
```

6.3 Screen Objects and Actions

The interface prompts the user with a number of different input objects. Here are the prompts given:

- File Name Input - The user enters the name of one or more CSV files containing ballots.
- Number of Seats - The user enters the number of seats to fill for the election, which must be greater than 0 and less than the number of candidates on each ballot.
- Voting Algorithm - The system prompts the user to select an algorithm: “1” for STV or “2” for Plurality.
- Disable Shuffle (STV only) - If STV is selected, the user is prompted to disable ballot shuffling for testing purposes: “1” for yes, “0” for no.
- Help - At any point during the process, the user can input “H” to open the help window.
- Close Program - The user enters if they want to terminate the program or not: “1” for yes, “0” for no.

The user then takes the action of typing a response for each of these prompts. The system validates each input and stores information in variables accordingly.

7. Requirements Matrix

Functional Requirement	System Component	Type
4.1 REQ-1: The user is prompted for input of the file name.	IElection	Class(es)
4.1 REQ-2: The system contains every single ballot that will be used to complete the election	Vector<Ballot> ballots in IElection class	Data
4.2 REQ-1: The user is prompted for input of the number of seats.	IElection	Class(es)
4.2 REQ-2: The system stores an int number of seats, 0 < n < # of candidates	numSeats in IElection class	Data
4.3 REQ-1: The user is prompted for input of the election type.	IElection	Class(es)
4.3 REQ-2: The system stores the algorithm type as a boolean.	IElection	Class(es)
4.4 REQ-1: The user is prompted for the number of ballots and candidates.	IElection	Class(es)
4.4 REQ-2: The system stores ballots and candidates as variables	Vector<Ballot> ballots in IElection class, candidateRanking in the Ballot class	Data
4.4 REQ-3: There are 0 < n ballots and 1 < n candidates.	candidateRanking in Ballot class	Data
4.5 REQ-1: The user can input ‘h’ at any point in the program, except for the running of an	HelpWindow	Class(es)

election.		
4.5 REQ-2: The system will provide a help window to the user.	HelpWindow	Class(es)
4.5 REQ-3: The user can exit out of this window back to their previous menu at any time.	HelpWindow	Class(es)
4.6 REQ-1: The system propagates election results and statistics to be displayed on screen.	IElection	Class(es)
4.7 REQ-1: The system will read in the ballots.	BallotHandler, Ballot	Class(es)
4.7 REQ-2: The system will run the plurality voting algorithm on those ballots.	PluralityElection	Class(es)
4.7 REQ-3: The system will store a winner.	PluralityElection	Class(es)
4.8 REQ-1: The system will read in the ballots.	BallotHandler, Ballot	Class(es)
4.8 REQ-2: The system will run the STV voting algorithm on those ballots.	STVElection	Class(es)
4.8 REQ-3: The system will store a winner.	STVElection	Class(es)
4.9 REQ-1: The system will create an audit file as a .txt.	AuditReport	Class(es)
4.10 REQ-1: The user will be given a choice to shuffle the ballots in the STV algorithm.	BallotHandler	Class(es)
4.10 REQ-2: If the user chooses not to shuffle the ballots, the ballots will remain	BallotHandler	Class(es)

unshuffled.		
4.10 REQ-3: If the user chooses to shuffle the ballots, the ballots will be shuffled.	BallotHandler	Class(es)

8. APPENDICE

N/A